Cuaderno de Ejercicios: Entrevista Técnica Accenture

Puesto: Analytics & Modeling Senior Analyst (Accenture), para el equipo de tecnología de RR.HH. de Mercado Libre. **Objetivo:** saber hacerlo a mano **y** saber guiar a un junior que se traba.

Cómo usar este cuaderno

Cada tema tiene 4 partes:

1. **Explicación simple.** Qué es, con palabras de todos los días. Cada término técnico lleva su explicación entre paréntesis.
2. **Dónde lo usaste vos.** Lo conectamos con Ciudad, Babilonia (Tomi), Sonner o COSME. En la entrevista, **un ejemplo propio vale más que una definición**.
3. **Ejercicios (3 o más).** Tapá la solución, resolvelo en voz alta y después compará.
4. **Si un junior se traba.** Cómo lo guiarías. Esto es lo que más evalúan para este puesto.

> **Regla de oro:** en la técnica, pensá en voz alta. Les importa más **cómo razonás** que llegar perfecto al resultado.

Tus proyectos en una tabla (memorizala)

Proyecto	Qué es	Stack (tecnologías)	Lo técnico para lucirte
Ciudad	Plataforma para una inmobiliaria: propiedades, contratos, pagos, liquidaciones a propietarios y un agente de IA interno	Python + FastAPI (framework para APIs), SQLAlchemy (ORM: escribir SQL con objetos Python), Postgres en Supabase, React, Claude (Anthropic), Docker en Easypanel	Agente con *tool calling* (el modelo usa herramientas que consultan la base); extracción de contratos PDF → datos con Claude; pipeline de índices IPC (INDEC) e ICL (BCRA) con caché y fallback; cron diario en GitHub Actions con reintentos; roles (admin, operador, finanzas, agente_ia)
Babilonia (Tomi)	Plataforma de métricas y conocimiento para Tomi, un bot de WhatsApp que asiste a asesores de seguros de Allianz	FastAPI, Postgres + pgvector (búsqueda por significado), OpenAI (gpt-4.1-mini + embeddings), **n8n** (14 workflows), WATI, Notion, Telegram	**RAG** (chunks de 1000 caracteres con 150 de solapamiento); humano en el loop (las correcciones aprobadas en el sandbox vuelven al RAG); *circuit breaker*; *rate limit*; caché; alertas sin duplicados; tabla de fallidos (tipo DLQ); métricas con SQL avanzado; tests
Sonner	Asistente de IA multicanal (web, Telegram, WhatsApp) para un negocio de eventos: agente para clientes, agente interno, contratos, analista de datos	FastAPI async, Postgres 16 + pgvector, **Redis**, Claude Sonnet/Haiku, embeddings de OpenAI, Google Calendar/Drive/Gmail	**Multi-agente** (un agente principal que delega en subagentes); webhooks con verificación (hmac); *debounce* con Redis (juntar ráfagas de mensajes); filtro de salida para no filtrar datos internos; endpoint "analista" que arma métricas con SQL y se las da a Claude
COSME	Motor que ejecuta herramientas para agentes de IA	Node.js + Express, Claude *function calling*, Google Sheets, OAuth (HubSpot, Meta, Stripe)	Credenciales cifradas con AES-256; refresco automático de tokens; reintentos con *backoff* exponencial

Frase puente para la entrevista

> "No lo vi solo en un curso: en Babilonia armé un RAG con pgvector y humano en el loop, en Sonner un sistema multi-agente con Redis y webhooks, y en Ciudad un pipeline que trae índices del BCRA y el INDEC con caché y fallback. Eso es lo que quiero llevar al equipo de RR.HH. y enseñárselo a los juniors."

---

1. SQL (y BigQuery)

Explicación simple

SQL es el idioma para hacerle preguntas a tablas.

- `SELECT`: qué columnas querés.
- `FROM`: de qué tabla.
- `WHERE`: filtro de filas.
- `GROUP BY`: agrupar.
- `HAVING`: filtro de grupos.
- `ORDER BY`: ordenar.

JOIN. Unir tablas por una columna en común (como un BUSCARV de Excel).

- `INNER JOIN` trae solo las filas que tienen pareja en las dos tablas.
- `LEFT JOIN` trae todo lo de la izquierda; si no hay pareja, completa con `NULL` (vacío).

Window function (función de ventana). Calcula mirando otras filas **sin colapsarlas** como hace un GROUP BY. Ejemplo: numerar dentro de cada grupo con `ROW_NUMBER() OVER (PARTITION BY area ORDER BY sueldo DESC)`.

CTE. `WITH nombre AS (...)` es una subconsulta con nombre. Hace que un query largo se lea como pasos.

Upsert (insertar o actualizar). `INSERT ... ON CONFLICT DO UPDATE` en Postgres, o `MERGE` en BigQuery. Si el dato ya existe, lo actualiza en vez de duplicarlo. Es la base de la **idempotencia** (correrlo dos veces da lo mismo que una vez).

BigQuery es la base de datos analítica de Google. Cobra por **bytes leídos**, así que:

- Nunca uses `SELECT *`.
- Filtrá por la columna de **partición** (la tabla está cortada por fecha).
- Usá **clustering** (ordenar los datos para que lea menos).

Dónde lo usaste vos

- **Babilonia:**
  - `db/schema.sql` crea índices compuestos como `(wa_id, created_at desc)`. Un índice es como el índice de un libro: encuentra rápido sin leer todo.
  - `routers/metrics.py` calcula tiempos de respuesta con funciones de ventana y series de tiempo con `date_trunc` (redondear fechas al día o la semana).
  - Hace upserts con `ON CONFLICT`.
- **Ciudad:** unos 20 modelos (tablas) en SQLAlchemy: Contrato, Pago, Garante, AjusteContrato, más una tabla de auditoría (AccionHistorial).
- **Sonner:** `routers/metrics.py` usa SQL crudo con `text()` para las métricas.

Ejercicios

Ejercicio 1.1: GROUP BY + HAVING. Tabla `licencias(empleado_id, tipo, fecha)`. Traé los empleados con **más de 2 licencias en el último año**.

```
SELECT empleado_id, COUNT(*) AS cantidad
FROM licencias
WHERE fecha >= CURRENT_DATE - INTERVAL '1 year'
GROUP BY empleado_id
HAVING COUNT(*) > 2
ORDER BY cantidad DESC;
```

> **Clave:** `WHERE` filtra filas **antes** de agrupar; `HAVING` filtra **grupos** después. En BigQuery la fecha se escribe `DATE_SUB(CURRENT_DATE(), INTERVAL 1 YEAR)`.

Ejercicio 1.2: el top 1 por grupo (window function). Tabla `empleados(id, nombre, area, sueldo)`. Traé el empleado **mejor pago de cada área**.

```
WITH ranking AS (
  SELECT area, nombre, sueldo,
         ROW_NUMBER() OVER (PARTITION BY area ORDER BY sueldo DESC) AS rn
  FROM empleados
)
SELECT area, nombre, sueldo
FROM ranking
WHERE rn = 1;
```

> **Clave:** `PARTITION BY` = "por cada área"; `ORDER BY` = el criterio del ranking.
>
> - Si hay empates y querés traer a todos los empatados, usá `RANK()`.

Ejercicio 1.3: lo que falta (LEFT JOIN + NULL). Tablas `empleados(id, nombre)` y `evaluaciones(id, empleado_id, periodo)`. Traé los empleados **sin evaluación en 2026**.

```
SELECT e.id, e.nombre
FROM empleados e
LEFT JOIN evaluaciones ev
       ON ev.empleado_id = e.id
      AND ev.periodo = '2026'
WHERE ev.id IS NULL;
```

> **Trampa típica:** si ponés `ev.periodo = '2026'` en el `WHERE` en vez de en el `ON`, el LEFT JOIN se convierte en INNER y no aparece nadie.
>
> - Esto es de lo que más le pasa a un junior.

Ejercicio 1.4: tiempo de respuesta promedio (como en Babilonia). Tabla `mensajes(conversacion_id, autor, created_at)`, donde `autor` es `'usuario'` o `'bot'`. Calculá el

tiempo promedio (en segundos) entre un mensaje del usuario y la respuesta siguiente del bot, **por día**.

```
WITH pares AS (
  SELECT conversacion_id, autor, created_at,
         LEAD(autor)      OVER (PARTITION BY conversacion_id ORDER BY created_at) AS
sig_autor,
         LEAD(created_at) OVER (PARTITION BY conversacion_id ORDER BY created_at) AS
sig_fecha
  FROM mensajes
)
SELECT date_trunc('day', created_at) AS dia,
       AVG(EXTRACT(EPOCH FROM sig_fecha - created_at)) AS seg_promedio
FROM pares
WHERE autor = 'usuario' AND sig_autor = 'bot'
GROUP BY 1
ORDER BY 1;
```

> **Clave:** `LEAD` mira la **fila siguiente** (y `LAG`, la anterior). `EXTRACT(EPOCH FROM intervalo)` pasa una diferencia de tiempo a segundos.

Ejercicio 1.5: carga sin duplicados (upsert). Todos los días llega un archivo con la nómina (`empleado_id, area, sueldo`). Cargalo **sin duplicar** si el proceso corre dos veces.

```
INSERT INTO empleados (id, area, sueldo)
VALUES (:id, :area, :sueldo)
ON CONFLICT (id) DO UPDATE
SET area = EXCLUDED.area,
    sueldo = EXCLUDED.sueldo;
```

En BigQuery, lo mismo se hace con `MERGE`:

```
MERGE empleados t
USING staging s ON t.id = s.id
WHEN MATCHED THEN UPDATE SET area = s.area, sueldo = s.sueldo
WHEN NOT MATCHED THEN INSERT (id, area, sueldo) VALUES (s.id, s.area, s.sueldo);
```

Si un junior se traba con SQL

- **"Me da filas duplicadas después del JOIN."** Le pido que cuente filas antes y después del JOIN (`COUNT(*)`). Casi siempre la tabla de la derecha tiene **varias filas por clave** (relación uno a muchos). Lo guío a agregar primero (`GROUP BY`) o a elegir una fila con `ROW_NUMBER`.
- **"El query en BigQuery sale caro o es lento."** Miramos juntos los bytes estimados que va a leer. Sacamos el `SELECT *`, filtramos por la partición de fecha y, si se repite mucho, armamos una tabla resumen.
- **Método para enseñar:** que escriba el query **por pasos con CTEs** y verifique el resultado de cada paso. No un monstruo de 80 líneas de una.

---

2. Python y pandas

Explicación simple

Python es el lenguaje para automatizar.

- **Variables:** cajitas con nombre.
- **Listas** `[1, 2, 3]`: se cuentan desde 0.
- **Diccionarios** `{"nombre": "Ana"}`: clave y valor, como un JSON.
- **if / for:** decisiones y repeticiones, con sangría (los espacios al inicio definen qué va adentro).
- **Funciones** con `def`: bloques reutilizables.
- **try/except:** atrapar errores sin que el programa se caiga.

pandas es "Excel con código". Un **DataFrame** es una tabla.

- `read_csv`: leer un archivo.
- `df[df["area"] == "IT"]`: filtrar.
- `groupby`: agrupar.
- `merge`: unir dos tablas (el JOIN de pandas).
- `fillna`: completar vacíos.
- `apply`: aplicar una función a cada fila.

Dónde lo usaste vos

- **Ciudad:** `services/indices_service.py` trae el IPC y el ICL, los guarda en caché y calcula el **factor de ajuste del alquiler**, contemplando que el INDEC publica con un mes de atraso. Es un cálculo de datos real con reglas de negocio.
- **Sonner:** `routers/analyst.py` arma agregados (totales, promedios) y se los pasa a Claude como contexto para que responda preguntas del negocio.
- **Ciudad:** genera contratos PDF y DOCX con Python (reportlab, python-docx).

Ejercicios

Ejercicio 2.1: limpieza de datos. `empleados.csv` tiene: `email` con mayúsculas y espacios, `sueldo` vacío en algunas filas y filas duplicadas. Limpialo.

```
import pandas as pd

df = pd.read_csv("empleados.csv")
df["email"] = df["email"].str.strip().str.lower()   # sacar espacios y pasar a
minúsculas
df = df.drop_duplicates(subset="email")             # un registro por email
df["sueldo"] = df["sueldo"].fillna(df["sueldo"].median())  # vacío -> mediana
```

> **Pregunta senior:** ¿completar con la mediana es correcto? Depende.
>
> - A veces conviene **marcar el dato faltante** y no inventarlo.
> - Si dudás, preguntá: "¿para qué se va a usar este dato?".

Ejercicio 2.2: rotación por área (merge + groupby). Tenés `empleados(id, area)` y `bajas(empleado_id, fecha_baja)`. Calculá el **% de rotación** (gente que se fue sobre el total) por área.

```
emp = pd.read_csv("empleados.csv")
bajas = pd.read_csv("bajas.csv")

df = emp.merge(bajas, left_on="id", right_on="empleado_id", how="left")
df["se_fue"] = df["fecha_baja"].notna()   # True si tiene fecha de baja

rotacion = (df.groupby("area")["se_fue"]
              .mean()
              .mul(100)
              .round(1)
              .sort_values(ascending=False))
print(rotacion)
```

> **Clave:** `how="left"` mantiene a todos los empleados. El promedio de un booleano (True/False) da directamente el porcentaje.

Ejercicio 2.3: ajuste de alquiler (como en Ciudad). Un contrato arranca en enero con alquiler de $100.000 y se ajusta **cada 3 meses** por inflación acumulada. Dado un diccionario de IPC mensual, calculá el nuevo valor.

```
ipc = {"2026-01": 2.2, "2026-02": 2.4, "2026-03": 3.7}  # % mensual

def ajustar(monto: float, meses: list[str]) -> float:
    factor = 1.0
    for mes in meses:
        if mes not in ipc:
            raise ValueError(f"Falta el IPC de {mes}")  # no inventar el dato
        factor *= 1 + ipc[mes] / 100
    return round(monto * factor, 2)

print(ajustar(100_000, ["2026-01", "2026-02", "2026-03"]))  # 108524.95
```

> **Lo senior:** la inflación se **compone**: se multiplica, no se suma (2,2 + 2,4 + 3,7 no es lo mismo).
>
> - Si falta un dato, mejor fallar explícito que calcular mal en silencio.
> - En Ciudad, ese caso lo resolví con caché y un valor de respaldo (fallback).

Ejercicio 2.4: leer un JSON anidado. Una API devuelve `{"data": [{"empleado": {"id": 1, "area": {"nombre": "IT"}}}]}`. Pasalo a una tabla plana.

```
import pandas as pd

respuesta = {"data": [{"empleado": {"id": 1, "area": {"nombre": "IT"}}}]}
df = pd.json_normalize(respuesta["data"])
# columnas: empleado.id, empleado.area.nombre
```

Si un junior se traba con Python

- **"Me tira `KeyError`."** Le pido que imprima `df.columns` o `dict.keys()`. El 90% de las veces el nombre de la columna está mal escrito o tiene un espacio de más.
- **"El merge me multiplicó las filas."** Igual que en SQL: hay claves repetidas. `df["id"].duplicated().sum()` lo muestra.
- **Enseñar a debuggear:**
  - Imprimir `df.head()` y `df.shape` (cantidad de filas y columnas) después de cada paso.
  - Usar el debugger del editor (pausar el programa y mirar las variables).

---

3. APIs, webhooks y colas

Explicación simple

API. Un mozo: le pedís algo al servidor y te trae la respuesta.

- **Verbos:** `GET` leer · `POST` crear · `PUT/PATCH` modificar · `DELETE` borrar.
- **Códigos de respuesta:**
  - **2xx** salió bien.
  - **4xx** el error es tuyo: 400 pedido mal armado, 401 sin credenciales, 403 sin permiso, 404 no existe, 429 demasiados pedidos.
  - **5xx** el error es del servidor.

Autenticación (demostrar quién sos):

- **API key:** una clave fija.
- **Bearer token:** `Authorization: Bearer ...`.
- **OAuth:** el "Entrar con Google"; da tokens que vencen y se refrescan.

Paginación. Los resultados vienen por páginas y hay que recorrerlas todas.

Rate limit. Límite de pedidos por minuto. Si te pasás recibís un **429**, y lo correcto es reintentar con **backoff exponencial**: esperar 1s, 2s, 4s...

Webhook. El otro sistema **te avisa** con un POST a tu URL (el timbre), en vez de que vos preguntes cada 5 minutos (*polling*).

- Hay que **verificar** que el aviso venga de quien dice venir, con una firma o un secreto.

Cola de mensajes (RabbitMQ, Pulsar, Pub/Sub). La fila del banco:

- El productor deja mensajes y el consumidor los procesa a su ritmo.
- **ACK:** "ya lo procesé".
- **DLQ** (*dead letter queue*): los mensajes que fallaron varias veces van a una fila aparte.

Dónde lo usaste vos

- **COSME:**
  - OAuth con HubSpot, Meta y Stripe, con refresco automático de tokens.
  - Reintentos con backoff exponencial.
- **Ciudad:**
  - Consume las APIs del INDEC (datos.gob.ar) y del BCRA, con caché y fallback si se caen.
  - WhatsApp vía Meta Graph y sincronización con Tokko Broker.
- **Sonner:**
  - Webhooks de WhatsApp (verificación con `hub.challenge`) y de Telegram (verificación del secreto con `hmac`).

    - El procesamiento corre en segundo plano (`BackgroundTasks`), así el webhook responde rápido.
- **Babilonia:** una tabla de envíos fallidos (`tomi_failed_dispatches`), que cumple el mismo rol que una DLQ.

Ejercicios

Ejercicio 3.1: traer todo de una API paginada con reintentos.

```
import requests, time

def traer_todo(url: str, token: str) -> list:
    datos, pagina = [], 1
    while True:
        for intento in range(5):
            r = requests.get(url, params={"page": pagina},
                             headers={"Authorization": f"Bearer {token}"},
                             timeout=10)
            if r.status_code not in (429, 500, 502, 503):
                break
            time.sleep(2 ** intento)          # 1, 2, 4, 8, 16 segundos
        r.raise_for_status()                  # si sigue mal, que explote con el error
        items = r.json()["items"]
        if not items:
            return datos
        datos.extend(items)
        pagina += 1
```

Detalles que suman puntos:

- **`timeout`:** no quedarse colgado para siempre.
- **Qué se reintenta:** solo los errores **transitorios** (429 y 5xx). Un 400 o un 401 no se arreglan reintentando.

Ejercicio 3.2: webhook seguro e idempotente. Llega un webhook de "nuevo empleado". El proveedor firma el cuerpo con HMAC-SHA256 en el header `X-Signature`. A veces manda el mismo evento dos veces. Escribí el endpoint.

```python
import hmac, hashlib
from fastapi import FastAPI, Request, HTTPException, BackgroundTasks

app = FastAPI()
SECRETO = b"viene-de-una-variable-de-entorno"
procesados = set()   # en producción: una tabla con el event_id como clave única

@app.post("/webhooks/empleados")
async def recibir(request: Request, tareas: BackgroundTasks):
    cuerpo = await request.body()
    esperada = hmac.new(SECRETO, cuerpo, hashlib.sha256).hexdigest()
    if not hmac.compare_digest(esperada, request.headers.get("X-Signature", "")):
        raise HTTPException(401, "firma inválida")
    evento = await request.json()
    if evento["event_id"] in procesados:      # idempotencia: ya lo hice
        return {"ok": True, "duplicado": True}
    procesados.add(evento["event_id"])
    tareas.add_task(dar_de_alta, evento)       # responder rápido, procesar después
    return {"ok": True}
```

Por qué `compare_digest`: compara en tiempo constante y evita ataques que adivinan la firma midiendo cuánto tarda la comparación.

- Lo usaste en Ciudad (`routers/ajustes.py`) y en Sonner.

Ejercicio 3.3: diseñar una cola con DLQ. El sistema de RR.HH. tiene que mandar 50.000 recibos de sueldo por mail el día 1. Diseñalo.

Respuesta modelo:

1. Un proceso **publica** un mensaje por recibo en una cola.
2. Varios **consumidores** los toman de a uno, mandan el mail y dan **ACK**.
3. Si falla, se reintenta con backoff; después de 3 intentos, el mensaje va a la **DLQ**, se alerta y alguien lo revisa.
4. Cada mensaje lleva un **id único**, para no mandar dos veces el mismo recibo.

Por qué cola y no un loop: si el proveedor de mail se cae a mitad de camino, no se pierde nada y se retoma donde quedó.

Ejercicio 3.4: REST vs GraphQL. ¿Cuándo usarías cada uno?

- **REST:** recursos simples, cacheable y estándar.
- **GraphQL:** el cliente elige exactamente qué campos traer. Sirve cuando las pantallas necesitan datos de muchos lugares y querés evitar diez llamadas.

Si un junior se traba con APIs

- **"La API me devuelve 401."** Revisamos juntos:

    - ¿El token venció?
    - ¿El header está bien escrito (`Bearer` con espacio)?
    - ¿Estamos en el ambiente correcto (prueba o producción)?
- **"Se duplican registros cuando llega el webhook."** Le explico que **los webhooks pueden llegar más de una vez**, por diseño. La solución es la idempotencia: guardar el id del evento.
- **Lo que mejoraría en mi propio código** (una respuesta honesta queda muy senior):
    - En Ciudad, las llamadas al BCRA usan `verify=False`, que apaga la verificación del certificado TLS (el candado del HTTPS). Hoy lo corregiría y se lo mostraría a un junior como ejemplo de lo que **no** hay que hacer.

---

4. LLMs, RAG y agentes

Explicación simple

LLM (*Large Language Model*). Un modelo que predice el siguiente pedazo de texto (Claude, GPT, Gemini).

- **Token:** pedacito de texto. Se paga por token.
- **Ventana de contexto:** cuánto texto ve el modelo de una vez.
- **Temperatura:** baja = respuestas predecibles (ideal para extraer datos); alta = más creativas.
- **Alucinación:** el modelo inventa con total seguridad.

Prompt engineering (saber pedir):

1. Rol.
2. Contexto.
3. Tarea.
4. Formato de salida (por ejemplo JSON).
5. Ejemplos (*few-shot*).
6. "Si no sabés, decí que no sabés".

RAG (*Retrieval-Augmented Generation*). Un examen a libro abierto:

1. Se cortan los documentos en pedazos (**chunks**).
2. Cada pedazo se convierte en un **embedding**: una lista de números que representa su significado.
3. Se guardan en una **base vectorial** (pgvector).
4. Ante una pregunta, se buscan los pedazos más parecidos y se pasan al modelo para que responda **con esa información**.

Tool calling / function calling. El modelo no ejecuta nada: **pide** usar una herramienta (por ejemplo `consultar_propiedades`), tu código la ejecuta y le devuelve el resultado.

Agente. Un LLM que en un loop decide qué herramienta usar hasta resolver la tarea. **Multi-agente:** un agente principal que delega en subagentes especializados.

Dónde lo usaste vos

- **Babilonia (RAG completo):**
  - Chunks de 1000 caracteres con 150 de **solapamiento** (para no cortar una idea a la mitad), embeddings `text-embedding-3-small` y pgvector.
  - Cada chunk tiene una etiqueta `source` y el bot filtra por ella.
  - **Humano en el loop:** un admin corrige respuestas en el sandbox y las aprobadas se suman al RAG (auto-aprendizaje).
- **Sonner (multi-agente):**

    - Un agente externo (Claude Sonnet) y uno interno (Haiku) que delegan en subagentes de calendario y memoria.
    - Un **filtro de salida** que bloquea palabras prohibidas para no filtrar datos internos.
- **Ciudad:**
    - Extracción de contratos: el PDF va a Claude y vuelve como datos estructurados.
    - Agente con herramientas que consultan la base.
- **COSME:** motor de function calling con Claude.

Ejercicios

Ejercicio 4.1: diseñar el chatbot de políticas de RR.HH. "¿Cuántos días de vacaciones me corresponden?" Diseñalo de punta a punta.

> **Respuesta modelo (5 pasos):**
>
> 1. **Aclarar:** ¿cuántos empleados? ¿en qué países (las políticas cambian por país)? ¿qué sistemas hay?
> 2. **Ingesta:** documentos de políticas → chunks (unos 1000 caracteres con solapamiento, como en Babilonia) → embeddings → pgvector, con metadatos (país, fecha de vigencia).
> 3. **Consulta:**
>   - Filtrar por el país del empleado, buscar los 5 chunks más parecidos y armar el prompt con esos chunks y la instrucción "respondé solo con esto y citá la fuente".
>   - Para lo personal ("**tus** días disponibles") usar **tool calling** a la API de RR.HH. con los permisos de ese empleado.
> 4. **Seguridad:** que nadie vea datos de otro, sin PII en los logs y un plan empresarial del proveedor.
> 5. **Calidad:**
>   - Un set de 50 preguntas con respuesta esperada.
>   - Pulgar arriba/abajo de los usuarios.
>   - Las correcciones aprobadas vuelven al RAG (lo mismo que hice en Babilonia).

Ejercicio 4.2: extraer datos de un CV con salida estructurada. Escribí el prompt y explicá cómo validás la salida.

```
Sos un asistente de RR.HH. Extraé los datos del CV de abajo.
Respondé SOLO un JSON con este formato:
{"nombre": str, "anios_experiencia": int, "skills": [str], "ultimo_puesto": str | null}
Si un dato no aparece, usá null. No inventes.
CV:
"""<texto del CV>"""
```

```
import json
from pydantic import BaseModel

class CV(BaseModel):
    nombre: str
    anios_experiencia: int | None
    skills: list[str]
    ultimo_puesto: str | None

datos = CV(**json.loads(respuesta_del_modelo))  # si no cumple el formato, falla y
reintentamos
```

Lo senior:

- Temperatura baja.
- **Validar con código** (pydantic revisa que el JSON tenga los tipos correctos).
- Reintentar si falla.
- Nunca dejar que la IA **decida** sola si alguien avanza en el proceso: humano en el loop.
- Es el mismo patrón de Ciudad (PDF de contrato → datos).

Ejercicio 4.3: el bot alucina. Usuarios reportan que el chatbot inventa políticas. ¿Qué hacés?

Respuesta modelo, en orden:

1. **Reproducir:** juntar ejemplos reales.
2. **Mirar qué chunks se recuperaron.** Si no está la información, el problema es la búsqueda, no el modelo: revisar el chunking, los metadatos y la cantidad de chunks.
3. **Ajustar el prompt:** "si no está en el contexto, decí que no sabés y derivá a RR.HH.".
4. Bajar la temperatura y **citar la fuente** en cada respuesta.
5. **Agregar esos casos al set de prueba** para que no vuelvan.

Ejercicio 4.4: ¿cuándo usar tool calling y cuándo RAG?

- **RAG:** conocimiento en **texto** que cambia poco (políticas, manuales).
- **Tool calling:** datos **vivos o personales** que están en un sistema (saldo de vacaciones, estado de un pedido).
- En Sonner y Ciudad usé los dos.

Ejercicio 4.5: costo y velocidad. El bot es lento y caro. ¿Qué tocás?

- Usar un **modelo más chico** para las tareas fáciles, como hice con Haiku para el agente interno de Sonner.
- Cachear respuestas frecuentes.
- Mandar menos contexto.
- **Debounce:** juntar los mensajes de una ráfaga en un solo turno, como en Sonner con Redis.

Si un junior se traba con LLMs

- **"El modelo no me devuelve JSON válido."** Le muestro tres cosas:
  - Bajar la temperatura.
  - Dar un ejemplo exacto del formato esperado.
  - Validar con pydantic y reintentar una vez.
- **"No sé si el cambio de prompt mejoró o empeoró."** Le enseño a armar un **set de evaluación**: 20 a 50 casos con respuesta esperada, que se corren antes y después del cambio. Sin eso, mejorar el prompt es adivinar.
- **Regla que le dejo:** primero fijarse si el modelo **tenía la información**; recién después culpar al modelo.

---

5. n8n y automatización

Explicación simple

n8n es una herramienta para armar automatizaciones uniendo **nodos** (cajitas) en un **workflow** (flujo). Es *open source* (código abierto) y se puede instalar en servidores propios.

- **Trigger:** lo que arranca el flujo.
  - **Webhook:** otro sistema te llama.
  - **Schedule:** a una hora fija.
  - **Evento de una app:** por ejemplo, "llegó un mail".
- **Nodos clave:**
  - **HTTP Request:** llamar cualquier API.
  - **IF / Switch:** decidir caminos.
  - **Code:** JavaScript o Python cuando lo visual no alcanza.
  - **Loop / Split in Batches:** procesar de a tandas.
  - **AI Agent:** usar un LLM dentro del flujo.
  - **Error Trigger:** se dispara cuando otro flujo falla.
- Los datos viajan como **lista de JSON**. Con `{{ $json.campo }}` usás un dato del paso anterior.

Dónde lo usaste vos

- **Babilonia:** 14 workflows de n8n (en la carpeta `n8n/`). Uno corre **cada 5 minutos** y vuelve a contactar las conversaciones que llevan 23 horas o más sin respuesta, antes de que se cierre la ventana de 24 horas de WhatsApp.
  - Además, una **tabla de lock** (bloqueo) evita que el mismo flujo se dispare dos veces a la vez.
- **Sonner:** el *debounce* con Redis (juntar ráfagas de mensajes) es un **patrón que pasaste de n8n a código** cuando el flujo creció. Ese criterio de cuándo pasar de n8n a código queda muy senior.

Ejercicios

Ejercicio 5.1: flujo de solicitud de licencias. Diseñá en n8n: un empleado pide una licencia por formulario → se valida → aprueba el líder → se registra en el sistema de RR.HH.

Respuesta modelo:

1. **Webhook** recibe el formulario.
2. **IF:** ¿faltan datos? Si faltan, se responde con error.
3. **HTTP Request** al sistema de RR.HH.: ¿le quedan días?
4. **AI Agent** (opcional) resume el pedido en una línea para el líder.
5. Mensaje de **Slack** al líder con los botones Aprobar/Rechazar. La respuesta vuelve por otro webhook.
6. **HTTP Request** para registrar la licencia y avisar al empleado.
7. Un **Error workflow** avisa por Slack o Telegram si algo falla, como hice en Babilonia con las alertas a Telegram.

Ejercicio 5.2: el flujo corre dos veces y duplica. ¿Cómo lo evitás?

- **Idempotencia:** antes de crear, buscar si ya existe (por id de solicitud).
- **Lock:** una tabla o una clave en Redis que diga "este proceso ya está corriendo", como la tabla de lock de Babilonia.

Ejercicio 5.3: procesar 10.000 empleados con una API que permite 100 pedidos por minuto.

- **Split in Batches** de 100 y **Wait** de 60 segundos entre tandas.
- Reintento automático (*retry on fail*) en el HTTP Request.
- Guardar los fallidos en una tabla para reprocesarlos.
- Si esto se vuelve algo diario y pesado, pasarlo a un pipeline en código (06 - Pipelines).

Ejercicio 5.4: ¿cuándo NO usar n8n?

- Lógica muy compleja, volúmenes muy grandes, necesidad de tests automatizados o equipos grandes trabajando sobre el mismo flujo.
- En esos casos, **código versionado en Git**.
- n8n es ideal para integrar rápido y validar con negocio.

Si un junior se traba con n8n

- **"El flujo falla y no sé dónde."** Abrimos la pestaña **Executions** (historial de ejecuciones) y miramos nodo por nodo qué entró y qué salió. Es el equivalente de imprimir variables.
- **"Mi expresión `{{ }}` da vacío."** Casi siempre el campo está anidado o se llama distinto. Lo vemos en la vista JSON del nodo anterior.
- **Buenas prácticas que le exijo:**

- Nombrar bien los nodos.
- Las credenciales nunca dentro del nodo.
- Exportar el workflow a Git.
- Un error workflow por proyecto.

---

6. Pipelines de datos (ETL)

Explicación simple

Un **pipeline** es una cañería de datos: **extraer** (de APIs o archivos) → **transformar** (limpiar, calcular) → **cargar** (en la base).

- **ETL vs ELT:**
  - **ETL:** transformás antes de cargar.
  - **ELT:** cargás crudo y transformás adentro del data warehouse (la base analítica, como BigQuery). Es lo más común hoy.
- **Batch vs streaming:**
  - **Batch:** por tandas, por ejemplo cada noche (como el colectivo).
  - **Streaming:** dato por dato, en tiempo real (como un taxi).
- **Apache Beam** es la "receta" del pipeline y **Dataflow** es la "cocina" de Google que la ejecuta.
- **Orquestador** (Airflow / Cloud Composer, o un cron): decide **cuándo** corre cada paso y en qué orden.
- **Idempotencia:** si el pipeline corre dos veces, el resultado es el mismo. Se logra con MERGE/upsert.
- **Backfill:** volver a correr el pipeline para fechas pasadas.

Dónde lo usaste vos

- **Ciudad:**
  - Un pipeline diario de ajustes de alquiler.
  - Un loop interno de 24 horas, más un **cron en GitHub Actions** (`.github/workflows/ajustes-diarios.yml`) como respaldo, que llama a `/api/ajustes/correr` con un token y **reintenta 3 veces**.
  - Es redundancia: si una de las dos vías falla, la otra cubre.
- **Ciudad:** el pipeline de índices (INDEC/BCRA) usa caché y fallback. Es el "E" (extraer) y el "T" (transformar) de un ETL.
- **Babilonia:** ingesta de documentos para el RAG (`scripts/ingest_productos_rag.py`): leer el PDF, cortar en chunks, generar embeddings y cargar. Es un ETL de texto.

Ejercicios

Ejercicio 6.1: pipeline diario de nómina. Todas las noches hay que traer los empleados del sistema de RR.HH. (una API) y dejarlos en BigQuery para los dashboards. Diseñalo.

Respuesta modelo:

1. **Extract:** script en Python que pagina la API con reintentos (ejercicio 3.1) y guarda el crudo en Cloud Storage con la fecha en el nombre.
2. **Load:** cargar a una tabla `staging` (temporal) en BigQuery.
3. **Transform:** `MERGE` de staging contra la tabla final, que es idempotente (ejercicio 1.5).
4. **Orquestar:** Cloud Composer o un cron, con **alertas** si falla y un **chequeo de calidad**: si hoy llegaron 30% menos empleados que ayer, avisar antes de pisar los datos.
5. **Backfill:** como cada corrida usa su fecha, se puede re-correr cualquier día.

Ejercicio 6.2: ¿batch o streaming? Decidí para: (a) un dashboard mensual de rotación, (b) un alerta de "empleado bloqueado en el sistema".

- **(a) Batch:** el dato se mira una vez por mes; streaming sería caro sin sentido.
- **(b) Streaming o evento:** tiene que avisar en minutos. Webhook o Pub/Sub → función → alerta.

Ejercicio 6.3: el pipeline duplicó los datos de ayer.

- **Causa probable:** corrió dos veces y usaba `INSERT` en vez de `MERGE`.
- **Arreglo:** borrar el día duplicado y reprocesarlo, pasar a MERGE con clave única y agregar un control de duplicados.
- **Post-mortem:** qué pasó, por qué y qué cambiamos. Siempre sin culpables.

Ejercicio 6.4: un paso del pipeline de Beam (para reconocer la sintaxis).

```
import apache_beam as beam

with beam.Pipeline() as p:
    (p
     | "Leer"     >> beam.io.ReadFromText("empleados.csv", skip_header_lines=1)
     | "Parsear"  >> beam.Map(lambda linea: linea.split(","))
     | "Activos"  >> beam.Filter(lambda f: f[3] == "activo")
     | "Guardar"  >> beam.io.WriteToText("activos"))
```

Cómo se lee: cada `|` es un paso y `>>` le pone nombre. Una **PCollection** es la colección de datos que viaja y una **PTransform** es cada transformación.

Si un junior se traba con pipelines

- **"El pipeline falla a la madrugada y nadie se entera."** Le enseño que todo pipeline tiene que tener **alerta y logs** desde el día uno, no "después".

- **"No sé si los datos están bien."** Chequeos simples:
  - Cantidad de filas contra ayer.
  - Columnas clave sin nulos.
  - Sin ids duplicados.
- **Enseñanza clave:** diseñar pensando que **va a fallar**: reintentos, idempotencia y la posibilidad de re-correr.

---

7. BI y dashboards

Explicación simple

- **KPI** (*Key Performance Indicator*): el número clave que mira el negocio (rotación, tiempo de contratación).
- **Métrica vs dimensión:**
  - La **métrica** es lo que medís (cantidad, promedio).
  - La **dimensión** es por qué lo cortás (área, mes, país).
- **Qué gráfico usar:**

Querés mostrar	Gráfico
Tendencia en el tiempo	Líneas
Comparar categorías	Barras
Un número clave	Tarjeta grande (KPI)
Relación entre dos variables	Dispersión
Partes de un total con pocas categorías	Barras apiladas (evitar la torta con muchas porciones)

- **Diseño:** arriba los KPIs, en el medio las tendencias, abajo el detalle. Filtros visibles.
- **Looker Studio** (gratis, conectado a BigQuery) vs **Tableau** (más potente y pago).

Dónde lo usaste vos

- **Babilonia:** toda la plataforma es de **métricas**:
  - Analítica de conversaciones del bot.
  - Tiempos de respuesta y series por día con `date_trunc`.
  - Revisión de respuestas.
- **Sonner:**
  - Página de métricas.
  - El **"analista"**: preguntás en lenguaje natural y Claude responde usando agregados SQL. Es BI conversacional.

Ejercicios

Ejercicio 7.1: KPIs del chatbot de RR.HH. Definí 5 KPIs.

> 1. % de preguntas resueltas sin derivar a una persona.
> 2. Satisfacción (pulgar arriba / total).
> 3. Tiempo promedio de respuesta.
> 4. Preguntas sin respuesta, por tema (dónde falta documentación).
> 5. Uso: empleados activos por semana.

Ejercicio 7.2: dashboard de rotación para un líder de RR.HH. ¿Cómo lo armás?

> - **Arriba:** tarjetas con rotación del mes, rotación anual y bajas del mes.
> - **Medio:** línea de rotación por mes y barras de rotación por área.
> - **Abajo:** tabla de bajas con motivo.
> - **Filtros:** país, área y antigüedad.
> - **Fuente:** la tabla que alimenta el pipeline del ejercicio 6.1.
> - **Lo senior:** antes de armarlo, preguntar **qué decisión va a tomar** el líder con ese tablero.

Ejercicio 7.3: explicar un número raro. El dashboard muestra que la rotación de IT subió del 2% al 9% en un mes. ¿Qué hacés antes de reportarlo?

> 1. **Validar el dato:** ¿hay duplicados? ¿cambió la definición o la fuente?
> 2. **Abrir por dimensión:** ¿es un solo equipo? ¿un solo país?
> 3. **Buscar el contexto:** ¿hubo una reestructuración?
> 4. Recién ahí comunicarlo, **con la explicación**.

Si un junior se traba con dashboards

- **"Hice un tablero con 20 gráficos."** Le pregunto: "¿qué decisión toma el líder con esto?" Nos quedamos con lo que responde esa pregunta.
- **"Los números no coinciden con los de RR.HH."** Revisamos juntos las **definiciones** (¿quién cuenta como empleado activo? ¿desde qué fecha?). Casi nunca es un error de cálculo: es una diferencia de definición.

---

8. Seguridad y datos personales (PII)

Explicación simple

PII (*Personally Identifiable Information*) son los datos que identifican a una persona: nombre, DNI, mail, sueldo, salud. En RR.HH. **todo** es sensible.

Mi regla de 4 para usar IA con datos personales:

1. **Enmascarar** antes de mandar al modelo. **Presidio** (de Microsoft) detecta los datos personales con su Analyzer y los reemplaza con su Anonymizer.
2. Mandar **lo mínimo** necesario.
3. Usar **planes empresariales** en los que el proveedor no entrena con tus datos.
4. **No guardar PII en los logs** (los registros del sistema).

Otros conceptos:

- **Secretos:** contraseñas y claves van en variables de entorno o en un gestor de secretos, **nunca en el código ni en un prompt**.
- **Mínimo privilegio:** cada usuario y cada sistema accede solo a lo que necesita.
- **Roles (RBAC):** permisos por rol.
- **Sesgo (bias):** un modelo puede discriminar (por ejemplo por género o edad). En selección, **la IA sugiere y una persona decide**.

Dónde lo usaste vos

- **COSME:** credenciales cifradas con **AES-256** (cifrado fuerte) y refresco de tokens OAuth.
- **Ciudad:**
  - Login con JWT y contraseñas con **bcrypt** (no se guardan en texto plano).
  - **Roles** admin, operador, finanzas y agente_ia.
  - `compare_digest` para validar el token del cron.
- **Sonner:**
  - Verificación hmac de los webhooks.
  - Lista de bloqueo de contactos.
  - Filtro de salida del agente.
  - La tabla de contratos guarda **DNI**: es PII y hay que tratarla como tal.

Ejercicios

Ejercicio 8.1: code review de seguridad (encontrá los 5 problemas).

```
OPENAI_KEY = "sk-abc123..."                                  # 1

@app.get("/empleados/{id}")
def ver_empleado(id: int):                                   # 2
    q = f"SELECT * FROM empleados WHERE id = {id}"           # 3
    emp = db.execute(q).fetchone()
    log.info(f"Consultado: {emp}")                           # 4
    r = requests.get(URL_BCRA, verify=False)                 # 5
    return emp
```

Problemas:

1. **Clave en el código:** si se sube a Git, queda expuesta para siempre. Va en una variable de entorno o un gestor de secretos, y hay que **rotarla** (cambiarla), porque ya se filtró.
2. **Sin autenticación ni control de permisos:** cualquiera ve a cualquier empleado. Hay que exigir login y verificar que el que pide tenga permiso sobre **ese** empleado.
3. **SQL injection** (inyección SQL): armar el query concatenando texto permite que alguien meta SQL propio. Se usan **parámetros**: `text("... WHERE id = :id")`.
4. **PII en los logs:** se logea el registro completo. Hay que logear solo el id.
5. **`verify=False`:** apaga la verificación del certificado TLS. Fue un error real en Ciudad, contalo como aprendizaje.

Extra: `CORS allow_origins=["*"]` (permitir llamadas desde cualquier sitio web) en Sonner también lo corregiría, limitándolo a los dominios propios.

Ejercicio 8.2: mandar CVs a un LLM sin exponer datos.

```
from presidio_analyzer import AnalyzerEngine
from presidio_anonymizer import AnonymizerEngine

texto = "Juan Pérez, DNI 30.123.456, juan@mail.com, 8 años en Python"
hallazgos = AnalyzerEngine().analyze(text=texto, language="es")
limpio = AnonymizerEngine().anonymize(text=texto, analyzer_results=hallazgos).text
# -> "<PERSON>, DNI <ID>, <EMAIL_ADDRESS>, 8 años en Python"
```

Nota: para español, Presidio necesita un modelo de idioma en español y reglas propias para el DNI. Detectar PII no es perfecto, así que conviene sumar **revisión y pruebas**.

Ejercicio 8.3: "¿Y si no tenés Azure?"

"Los conceptos son los mismos, cambia el nombre del servicio":

- **LLM:** Azure OpenAI ↔ AWS Bedrock ↔ Google Vertex AI.
- **Secretos:** Key Vault ↔ Secrets Manager ↔ Secret Manager.
- **Colas:** Service Bus ↔ SQS ↔ Pub/Sub.
- **Data warehouse:** Synapse ↔ Redshift ↔ BigQuery.

Yo trabajé con Anthropic, OpenAI y Supabase; me adapto al stack del cliente.

Si un junior se traba con seguridad

- **"Subí una clave a GitHub por error."** Sin retarlo, pero rápido:
  1. Rotar la clave **ya**.
  2. Sacarla del código y pasarla a una variable de entorno.
  3. Agregar un escáner de secretos al CI.
  4. Contarlo en la retro como aprendizaje del equipo.
- **Cultura que quiero dejar:** checklist de seguridad en cada PR, con estas preguntas:
  - ¿Hay secretos en el código?
  - ¿Hay PII en los logs?
  - ¿El endpoint pide permisos?
  - ¿Las consultas usan parámetros?

---

9. Confiabilidad (que el sistema no se caiga)

Explicación simple

- **Retry con backoff:** reintentar esperando cada vez más.
- **Circuit breaker** (disyuntor, como la térmica de tu casa). Si un servicio externo falla muchas veces seguidas, se **corta** un rato y no se lo sigue llamando. Tiene 3 estados:
  - **Cerrado:** todo normal.
  - **Abierto:** cortado, se responde con error rápido o con un plan B.
  - **Semiabierto:** se prueba de a poco si el servicio volvió.
- **Rate limiting:** limitar cuántos pedidos acepta tu sistema por minuto.
- **Caché:** guardar resultados para no recalcularlos.
  - **TTL:** tiempo de vida del dato guardado.
  - **LRU:** cuando se llena, se borra lo menos usado.
- **Alertas:** avisar cuando algo falla, **sin spam**: agrupar los errores repetidos.
- **Healthcheck:** un endpoint que responde "estoy vivo" para que la infraestructura sepa si reiniciar el servicio.

Dónde lo usaste vos

Babilonia tiene todo el kit:

- **Circuit breaker** con los 3 estados y seguro para varios hilos a la vez (*thread-safe*): `services/tomi/circuit.py`. Envuelve al cliente de OpenAI en `openai_cb.py`.
- **Rate limit:** `ratelimit.py`.
- **Caché TTL/LRU:** `cache.py`.
- **Alertas a Telegram que no se repiten**, agrupadas por firma del error: `alertas.py`.

Además:

- **Sonner:** healthchecks en `docker-compose.yml`.
- **Ciudad:** el cron de respaldo con reintentos.

Ejercicios

Ejercicio 9.1: explicá un circuit breaker a un junior.

> "Es la térmica de tu casa. Si OpenAI falla 5 veces seguidas, el breaker se **abre** y durante 30 segundos ni lo llamamos: respondemos 'estoy con demoras, te derivo a una persona'. Pasado ese tiempo, pasa a **semiabierto**: deja pasar un pedido de prueba. Si anda, vuelve a **cerrado**; si no, se abre de nuevo. Así no saturamos un servicio caído ni dejamos al usuario esperando."

Ejercicio 9.2: pseudocódigo de un circuit breaker.

```
import time

class CircuitBreaker:
    def __init__(self, max_fallos=5, espera=30):
        self.fallos, self.max, self.espera = 0, max_fallos, espera
        self.abierto_desde = None

    def llamar(self, funcion, *args):
        if self.abierto_desde and time.time() - self.abierto_desde < self.espera:
            raise RuntimeError("Servicio cortado: usar plan B")    # ABIERTO
        try:
            resultado = funcion(*args)                              # CERRADO o
SEMIABIERTO
            self.fallos, self.abierto_desde = 0, None
            return resultado
        except Exception:
            self.fallos += 1
            if self.fallos >= self.max:
                self.abierto_desde = time.time()                    # se abre
            raise
```

Ejercicio 9.3: incidente en producción. El chatbot de RR.HH. deja de responder un lunes a las 9 AM. Contá qué hacés, minuto a minuto.

1. **Contener:** mensaje al usuario ("estamos con demoras") y avisar a los interesados.
2. **Diagnosticar:**
   - Mirar las alertas y los logs.
   - Ver si el problema es nuestro o del proveedor (la página de estado de OpenAI o Anthropic).
   - Ver si hubo un deploy (una subida de cambios) reciente.
3. **Mitigar:** revertir el último deploy (*rollback*) o activar el plan B (otro modelo, respuesta fija).
4. **Comunicar** cada 30 minutos, aunque no haya novedades.
5. **Post-mortem sin culpables:** causa raíz, qué alerta faltó y qué cambiamos para que no vuelva.

Si un junior se traba en un incidente

- **Le doy un rol claro:** "vos mirás los logs, yo hablo con negocio". En un incidente, la confusión de roles empeora todo.
- **Después, en frío:** repasamos el incidente juntos y él escribe el post-mortem. Así aprende más que de cualquier curso.

---

10. Liderazgo técnico y manejo de equipo (lo más importante)

Explicación simple

El puesto **no** es solo programar: es **multiplicar al equipo**, que los juniors resuelvan solos cada vez más.

Mi método para destrabar a un junior (4 pasos):

1. **"¿Qué probaste?"** Nunca dar la respuesta servida de entrada.
2. **Preguntas guía:** "¿Qué dice el error? ¿En qué línea? ¿Qué esperabas que pase?"
3. **Pair programming** (programar juntos en una pantalla) si sigue trabado más de 30 o 60 minutos.
4. **Documentar** la solución para el equipo: un README, una wiki o una charla corta.

Code review (revisar el código antes de sumarlo):

- Orden de revisión: ¿resuelve el problema? → ¿se entiende? → ¿tiene tests? → ¿es seguro y rinde bien?
- Explicar **el porqué** de cada comentario.
- Marcar qué es **bloqueante** y qué es un detalle (*nit*).
- Elogiar lo que está bien hecho.

Llevar un proyecto de A a B:

1. Entender el problema con el área (*stakeholder*).
2. Definir un **MVP** (lo mínimo que ya da valor).
3. Armar el **backlog** (lista priorizada de tareas).
4. Trabajar en sprints de 1 o 2 semanas.
5. Hacer demos frecuentes.
6. Cerrar cada sprint con una **retro** (qué mejorar).

Dónde lo usaste vos

- **Babilonia:**
  - Prompts **versionados** (`PROMPT_TOMMY_SYSTEM_v4.7.md`).
  - Guías de deploy (`DEPLOY.md`).
  - Tests que se saltean solos si faltan credenciales.
  - Todo eso **facilita que otro entre al proyecto**. Es lo que harías con los juniors.
- **Commits con formato** `feat(...)` / `fix(...)` (*Conventional Commits*): un historial que se entiende.
- **Ciudad:** el Dockerfile tiene comentarios que explican **por qué** se sacó `apt-get` (para bajar el tiempo de build). Documentar decisiones es enseñar.
- **Honestidad que suma:** Ciudad y Sonner **no tienen tests automatizados**. Lo podés contar así: "Lo aprendí: en un equipo con juniors, los tests son la red de seguridad. Por

eso en Babilonia ya los sumé y es lo primero que instalaría en el equipo."

Ejercicios (casos de manejo de equipo)

Caso 10.1: el junior trabado hace dos días. Un junior lleva 2 días con un bug en la integración con la API de RR.HH. y no avisó.

> **Respuesta modelo:**
>
> - **Primero, sin culpas:** "Contame qué viste y qué probaste".
> - **Destrabo con pair programming:** reproducimos el error y lo aislamos.
> - **Lo de fondo:** acordamos la **regla de los 60 minutos**: si estás trabado más de una hora, pedí ayuda. Pedir ayuda no es debilidad, es eficiencia.
> - Lo llevo a la daily como regla del equipo, no como reto personal.

Caso 10.2: PR que funciona pero está mal hecho. Un junior manda un PR que anda pero tiene un query armado con f-string (riesgo de SQL injection), sin tests y con una función de 200 líneas.

> **Respuesta modelo:**
>
> - No lo reescribo yo. Comento **en orden de importancia**:
>   1. **Bloqueante:** la inyección SQL, con un link y un ejemplo de cómo usar parámetros.
>   2. **Pedido:** un test del caso principal.
>   3. **Sugerencia:** partir la función.
> - Si son muchos comentarios, **mejor una llamada de 15 minutos** que 30 comentarios escritos.
> - Reconozco lo que está bien.

Caso 10.3: dos devs no se ponen de acuerdo. Uno quiere hacer el flujo en n8n y otro en Python.

> **Respuesta modelo:**
>
> - Escucho a los dos y bajo la discusión a **criterios**: volumen, complejidad, quién lo mantiene, necesidad de tests y tiempo de entrega.
> - Muchas veces la respuesta es **"n8n para validar rápido; si crece, a código"** (lo que hice con el debounce de Sonner).
> - Decido, dejo escrito el porqué y lo revisamos en un mes.

Caso 10.4: negocio pide algo para ayer. RR.HH. quiere el chatbot completo en 2 semanas y el equipo estima 6.

Respuesta modelo: negocio **alcance, no calidad**. "En 2 semanas llegamos con el MVP: vacaciones y licencias, para Argentina, con fuentes citadas. En las siguientes iteraciones sumamos otros países y los datos personales vía API."

- Muestro el plan con fechas y riesgos, y valido con ellos qué es lo más valioso.

Caso 10.5: onboarding de un junior nuevo. Entra un junior al proyecto del chatbot. ¿Qué hacés la primera semana?

1. **Día 1:** setup del entorno con una guía escrita (como el `DEPLOY.md` de Babilonia) y una explicación de la arquitectura en un dibujo de 15 minutos.
2. **Días 2 y 3:** un ticket chico y real (*good first issue*) con pair programming.
3. **Días 4 y 5:** su primer PR solo, con code review detallado.
4. Le pido que **mejore la guía de onboarding** con lo que le costó: así la guía mejora con cada persona que entra.

Caso 10.6: el equipo no hace tests. ¿Cómo instalás la práctica sin frenar las entregas?

- Empezar chico: **tests para todo bug que se arregle** (así no vuelve) y para lo nuevo.
- Sumar tests al **CI** (integración continua: se corren solos en cada PR).
- Dar el ejemplo en mis propios PRs.
- Como en Babilonia: tests que **no se rompen** si faltan credenciales, para que nadie los apague por molestos.

Caso 10.7: medir si el equipo mejora.

- Menos idas y vueltas por PR.
- Estimaciones más cercanas a la realidad.
- Menos bugs que vuelven.
- **Más autonomía:** los juniors preguntan cosas nuevas, no las mismas.
- Cada cierto tiempo, un 1:1 (charla individual) con cada uno.

Plantilla STAR para contar tus historias

STAR = Situación, Tarea, Acción, Resultado. Siempre terminá con un número.

	Babilonia (ejemplo armado con tu proyecto)
S	El bot de WhatsApp de Allianz respondía mal preguntas de productos y no había forma de corregirlo rápido
T	Armar una base de conocimiento confiable y un circuito para mejorarla
A	RAG con pgvector (chunks de 1000 con solapamiento de 150), sandbox donde un admin corrige respuestas y las aprobadas vuelven al RAG, circuit breaker y alertas a Telegram
R	*[completá con tu número real: % de respuestas correctas, tiempo de respuesta, cantidad de correcciones]*

> **Importante:** completá los **resultados con números reales** de tus proyectos. Si no los tenés medidos, decilo honestamente: "no lo medimos formalmente; lo que vimos fue...".

---

11. Caso integrador (diseño de sistema)

"Te sumás al equipo de tecnología de RR.HH. de Mercado Libre. Tenés 3 juniors. El primer proyecto es un asistente de IA para que los empleados consulten políticas y hagan trámites simples. ¿Cómo lo encarás?"

Respuesta modelo (5 minutos, en voz alta):

1. **Entender (semana 1):**
   - Reunión con RR.HH.: ¿qué preguntas llegan más? ¿cuántos empleados? ¿qué países? ¿qué sistemas hay?
   - Defino el **KPI** de éxito: % de consultas resueltas sin derivar a una persona.
2. **MVP (semanas 2 a 4):** solo políticas de vacaciones y licencias, para un país.
   - RAG sobre los documentos (como Babilonia).
   - Respuestas con fuente citada.
   - Canal: Slack o la intranet.
3. **Arquitectura:**
   - Backend en FastAPI.
   - Base vectorial (pgvector o Vertex AI Vector Search si están en GCP).
   - LLM con plan empresarial.
   - Presidio para la PII.
   - Logs sin datos personales.
   - Circuit breaker y alertas.
4. **Equipo:**
   - Junior A: ingesta de documentos.
   - Junior B: API y permisos.
   - Junior C: evaluación (set de preguntas) y dashboard de métricas.
   - Yo: arquitectura, code review, destrabar y la relación con RR.HH.
   - Daily de 15 minutos, demo cada 2 semanas.
5. **Iterar:**
   - Sumar tool calling para trámites (saldo de días).
   - Sumar más países.
   - Las correcciones de los usuarios vuelven al RAG.
6. **Riesgos:**
   - Datos sensibles: permisos por rol.
   - Alucinaciones: evaluación continua y "si no sabés, derivá".
   - Adopción: comunicar bien el lanzamiento y medir el uso.

12. Si te quedás en blanco + inglés

Si te quedás en blanco:

1. **Hacé una pregunta aclaratoria** (queda senior): "¿Para cuántos usuarios sería?", "¿En qué nube trabajan?"
2. **Pensá en voz alta:** "Lo encararía así: primero..., después...".
3. **Sé honesto:** "Eso puntual no lo usé, pero lo resolvería así... y lo validaría con...". **Nunca inventar.**
4. **Volvé a tu terreno:** "En Babilonia tuve algo parecido: ...".

En inglés (por si cambian de idioma):

- *"I'm a hybrid profile: technical background in Python, SQL, APIs and generative AI, plus experience leading projects end to end."*
- *"In Babilonia I built a RAG system with pgvector and a human-in-the-loop correction flow, with a circuit breaker and alerting for reliability."*
- *"When a junior gets stuck, I first ask what they've tried, guide them with questions, pair program if needed, and we document what we learned."*
- *"I'd start with an MVP, validate it with the business, and then scale."*
- *"Could you clarify the expected volume?"* (para ganar tiempo)

Preguntas para hacerles al final:

- ¿Qué stack usan hoy y en qué etapa se traban más los proyectos?
- ¿Cuántos devs serían y de qué nivel?
- ¿Cómo se mide el éxito del puesto a los 6 meses?